# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: OPERATING SYSTEMS** **COURSE CODE: CSA04**

---

**COURSE OUTCOME COVERED**

***CO2:*** Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

*Assessment Tool Weightage: 40%*

---

**QUESTIONS: 10** **Total Marks:100**

Annexure A

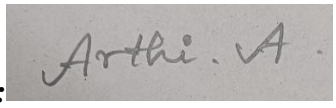
Analytical Problem Solving

---

***Code of Conduct:***

*I* **ARTHI A – 192525152** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

**Question 1: CPU Scheduling – Waiting Time & Turnaround Time Analysis**

A set of processes arrive at time 0 with the following burst times:

| Process | Burst Time (ms) |
|---|---|
| P1 | 10 |
| P2 | 4 |
| P3 | 6 |
| P4 | 2 |

a. Calculate Waiting Time and Turnaround Time for each process using SJF (Non-preemptive).
b. Compute the average waiting time and average turnaround time.
c. Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

**(10 Marks)**

**Question 2: Round Robin Scheduling – Time Quantum Impact**

Consider the following processes:

| Process | Burst Time (ms) |
|---|---|
| P1 | 20 |
| P2 | 5 |
| P3 | 13 |
| P4 | 8 |

Let the time quantum = 4 ms.

a. Construct the Gantt chart for Round Robin.
b. Calculate Waiting Time and Turnaround Time of each process.
c. Analyze how changing the quantum to 10 ms would affect system performance (CPU utilization & context switching).

**(10 Marks)**

**Question 3: Deadlock – Safety Check Using Banker's Algorithm**

A system has 3 resource types (A, B, C). Total resources:
A = 10, B = 5, C = 7.

The Allocation and Maximum Need matrices are:

Allocation Matrix

| Process | A | B | C |
|---|---|---|---|
| P1 | 0 | 1 | 0 |
| P2 | 2 | 0 | 0 |
| P3 | 3 | 0 | 2 |
| P4 | 2 | 1 | 1 |
| P5 | 0 | 0 | 2 |

Maximum Matrix

| Process | A | B | C |
|---|---|---|---|
| P1 | 7 | 5 | 3 |
| P2 | 3 | 2 | 2 |
| P3 | 9 | 0 | 2 |
| P4 | 4 | 2 | 2 |
| P5 | 5 | 3 | 3 |

a. Calculate the Need Matrix.
b. Determine if the system is in a safe state using Banker's Algorithm.
c. Provide a valid safe sequence if it exists.

**(10 Marks)**

**Question 4: Deadlock Detection – Resource Allocation Graph (RAG)**

A system has the following resource allocations:

- P1 is holding R1 and waiting for R2

- P2 is holding R2 and waiting for R3
- P3 is holding R3 and waiting for R1

a. Draw the Resource Allocation Graph (describe in text).
b. Determine whether the system is in deadlock.
c. Recommend a strategy (prevention OR avoidance OR recovery) to resolve it and justify the method.

**(10 Marks)**

**Question 5: CPU Utilization – Multiprogramming Level Analysis**

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

$$U = 1 - (p^n)$$

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.
b. Compute CPU utilization if the degree of multiprogramming increases to 6.
c. Interpret which configuration improves system performance and why.

**(10 Marks)**

**Question 6: Producer–Consumer Problem Using Semaphores**

A bounded buffer has a capacity of 6 items. Initially, the buffer is empty. Three producer operations occur followed by two consumer operations.

a. Show the values of the semaphores (empty, full, mutex) after each operation.
b. Explain how semaphores prevent race conditions.
c. What problem occurs if the mutex semaphore is removed? Justify your answer.

**(10 Marks)**

**Question 7: Process Synchronization – Critical Section**

Two processes increment a shared variable X initially equal to 100. Each process executes the increment operation 1000 times without synchronization.

a. Explain how a race condition occurs.
b. Determine the expected value of X with proper synchronization.
c. Suggest one hardware-based and one software-based synchronization solution.

**(10 Marks)**

**Question 8: Thread Scheduling and Performance**

A program creates 40 user-level threads on a system having 8 CPU cores. Each thread requires 12 ms CPU time and performs negligible I/O.

a. Compare the Many-to-One and One-to-One multithreading models.
b. Estimate how CPU utilisation differs between the two models.
c. Recommend the better threading model with justification.

**(10 Marks)**

**Question 9: Monitor Synchronization**

A monitor controls access to a shared printer used by multiple processes.

a. Explain how monitors ensure mutual exclusion.
b. Describe the role of condition variables wait() and signal().
c. Explain what may happen if signal() is omitted after a print job completes.

**(10 Marks)**

**Question 10: Deadlock Prevention vs Avoidance**

A system has four processes competing for two reusable resources.

a. Explain how deadlock prevention can eliminate one of the four necessary conditions for deadlock.
b. Using a suitable example, explain how deadlock avoidance differs from prevention.
c. Compare prevention and avoidance in terms of resource utilisation and system performance.

**(10 Marks)**

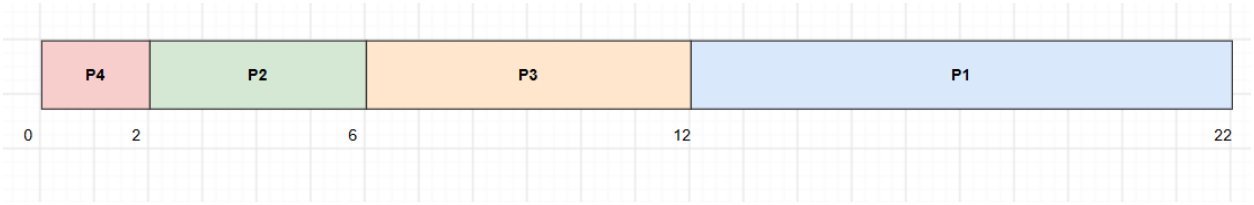
# Question 1: CPU Scheduling – SJF (Non-preemptive)

**Given:** All processes arrive at t = 0. Burst times: P1 = 10, P2 = 4, P3 = 6, P4 = 2.

**Method:** SJF non-preemptive picks the shortest burst first. Waiting Time (WT) = Start time − Arrival; Turnaround Time (TAT) = Completion − Arrival = WT + Burst.

**Execution order:** P4 → P2 → P3 → P1

**Gantt chart (draw.io: Q1_SJF_Gantt.drawio)**




| 0 – 2 | 2 – 6 | 6 – 12 | 12 – 22 |
|---|---|---|---|
| P4 | P2 | P3 | P1 |

**a. Waiting and Turnaround Time**

| Process | Burst | Completion | TAT = CT − 0 | WT = TAT − Burst |
|---|---|---|---|---|
| P4 | 2 | 2 | 2 | 0 |
| P2 | 4 | 6 | 6 | 2 |
| P3 | 6 | 12 | 12 | 6 |
| P1 | 10 | 22 | 22 | 12 |
| **Total** | | | **42** | **20** |

**b. Averages**

- Average WT = 20 / 4 = **5 ms**
- Average TAT = 42 / 4 = **10.5 ms**

**c. Interpretation (SJF vs FCFS)**

FCFS order P1 → P2 → P3 → P4 gives completion times 10, 14, 20, 22. So WT = 0, 10, 14, 20 (average **11 ms**) and TAT = 10, 14, 20, 22 (average **16.5 ms**).

| Metric | FCFS | SJF |
|---|---|---|
| Average WT | 11 ms | 5 ms |
| Average TAT | 16.5 ms | 10.5 ms |

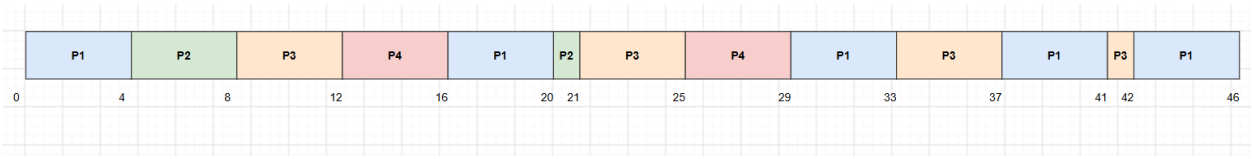
| Metric | FCFS | SJF |
|---|---|---|
| Total schedule length | 22 ms | 22 ms |
| CPU busy time | 22 ms (100%) | 22 ms (100%) |

SJF cuts average waiting time by about 55% and turnaround time by about 36%, because short jobs no longer sit behind the 10 ms job (the convoy effect). CPU utilization is identical (100%) in both, since there is no idle time and the total work is 22 ms. SJF therefore improves **responsiveness and throughput**, not raw CPU utilization. Its drawback is possible starvation of long jobs and the need to know burst times in advance.

# Question 2: Round Robin – Time Quantum Impact

**Given:** P1 = 20, P2 = 5, P3 = 13, P4 = 8, all arriving at 0. Quantum = 4 ms.

**a. Gantt chart (draw.io: Q2_RR_q4_Gantt.drawio)**




| 0–4 | 4–8 | 8–12 | 12–16 | 16–20 | 20–21 | 21–25 | 25–29 | 29–33 | 33–37 | 37–41 | 41–42 | 42–46 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P1 |

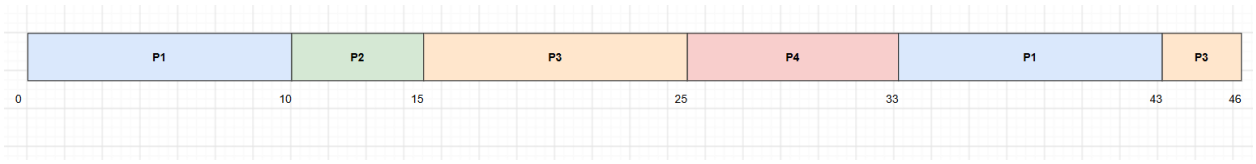
Remaining-time tracking: P1: 20→16→12→8→4→0; P2: 5→1→0; P3: 13→9→5→1→0; P4: 8→4→0.

**b. Waiting and Turnaround Time (quantum = 4)**

| Process | Burst | Completion | TAT | WT = TAT − Burst |
|---|---|---|---|---|
| P1 | 20 | 46 | 46 | 26 |
| P2 | 5 | 21 | 21 | 16 |
| P3 | 13 | 42 | 42 | 29 |
| P4 | 8 | 29 | 29 | 21 |
| **Average** | | | **34.5 ms** | **23 ms** |

Total time = 46 ms, with 13 time slices and therefore 12 context switches.

**c. Effect of changing the quantum to 10 ms (draw.io: Q2_RR_q10_Gantt.drawio)**




| 0–10 | 10–15 | 15–25 | 25–33 | 33–43 | 43–46 |
|---|---|---|---|---|---|
| P1 | P2 | P3 | P4 | P1 | P3 |

| Process | Completion | TAT | WT |
|---|---|---|---|
| P1 | 43 | 43 | 23 |
| P2 | 15 | 15 | 10 |
| P3 | 46 | 46 | 33 |
| P4 | 33 | 33 | 25 |
| **Average** | | **34.25 ms** | **22.75 ms** |

| Parameter | q = 4 ms | q = 10 ms |
|---|---|---|
| Time slices | 13 | 6 |
| Context switches | 12 | 5 |
| Average WT | 23 ms | 22.75 ms |
| Average TAT | 34.5 ms | 34.25 ms |
| Response time (first CPU access) | 0, 4, 8, 12 → avg 6 ms | 0, 10, 15, 25 → avg 12.5 ms |

**Analysis:** A larger quantum reduces context-switch overhead (12 → 5 switches), so more of the CPU time goes to useful work and effective CPU utilization rises slightly. Waiting and turnaround times are also marginally better here. The cost is a worse response time (6 → 12.5 ms), which hurts interactive jobs. If the quantum grows beyond the longest burst, RR degenerates into FCFS. If it is too small, the overhead of switching dominates. A good quantum is slightly longer than a typical CPU burst (commonly about 80% of bursts should finish within one quantum).

# Question 3: Banker's Algorithm – Safety Check

**Given:** Total A = 10, B = 5, C = 7.

**Available** = Total − Σ Allocation. Σ Allocation = (0+2+3+2+0, 1+0+0+1+0, 0+0+2+1+2) = (7, 2, 5). So **Available = (3, 3, 2)**.

**a. Need = Max − Allocation**

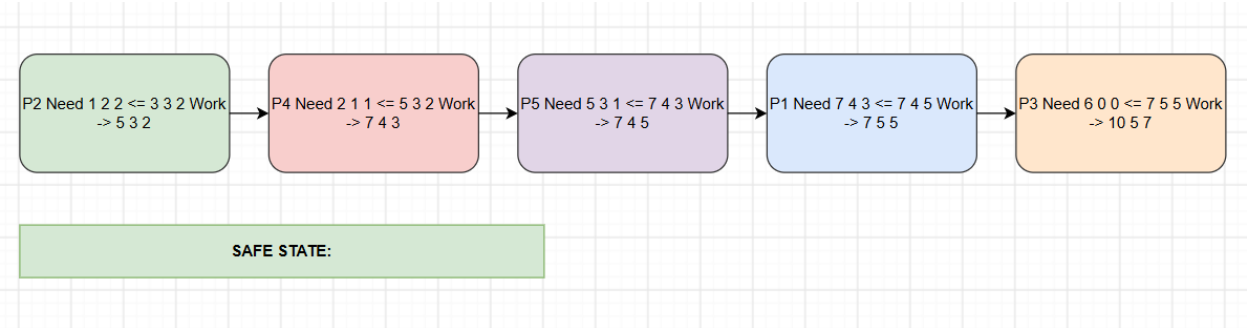
| Process | Allocation (A B C) | Max (A B C) | Need (A B C) |
|---|---|---|---|
| P1 | 0 1 0 | 7 5 3 | 7 4 3 |
| P2 | 2 0 0 | 3 2 2 | 1 2 2 |
| P3 | 3 0 2 | 9 0 2 | 6 0 0 |
| P4 | 2 1 1 | 4 2 2 | 2 1 1 |
| P5 | 0 0 2 | 5 3 3 | 5 3 1 |

**b. Safety algorithm (Work = Available = 3 3 2)**

| Step | Process chosen | Need ≤ Work? | Work after release (Work + Allocation) |
|---|---|---|---|
| 1 | P2 | (1 2 2) ≤ (3 3 2) ✓ | (5 3 2) |
| 2 | P4 | (2 1 1) ≤ (5 3 2) ✓ | (7 4 3) |
| 3 | P5 | (5 3 1) ≤ (7 4 3) ✓ | (7 4 5) |
| 4 | P1 | (7 4 3) ≤ (7 4 5) ✓ | (7 5 5) |
| 5 | P3 | (6 0 0) ≤ (7 5 5) ✓ | (10 5 7) |

All five processes can finish, and the final Work (10, 5, 7) equals the total resources, which confirms the calculation. **The system is in a SAFE state.**

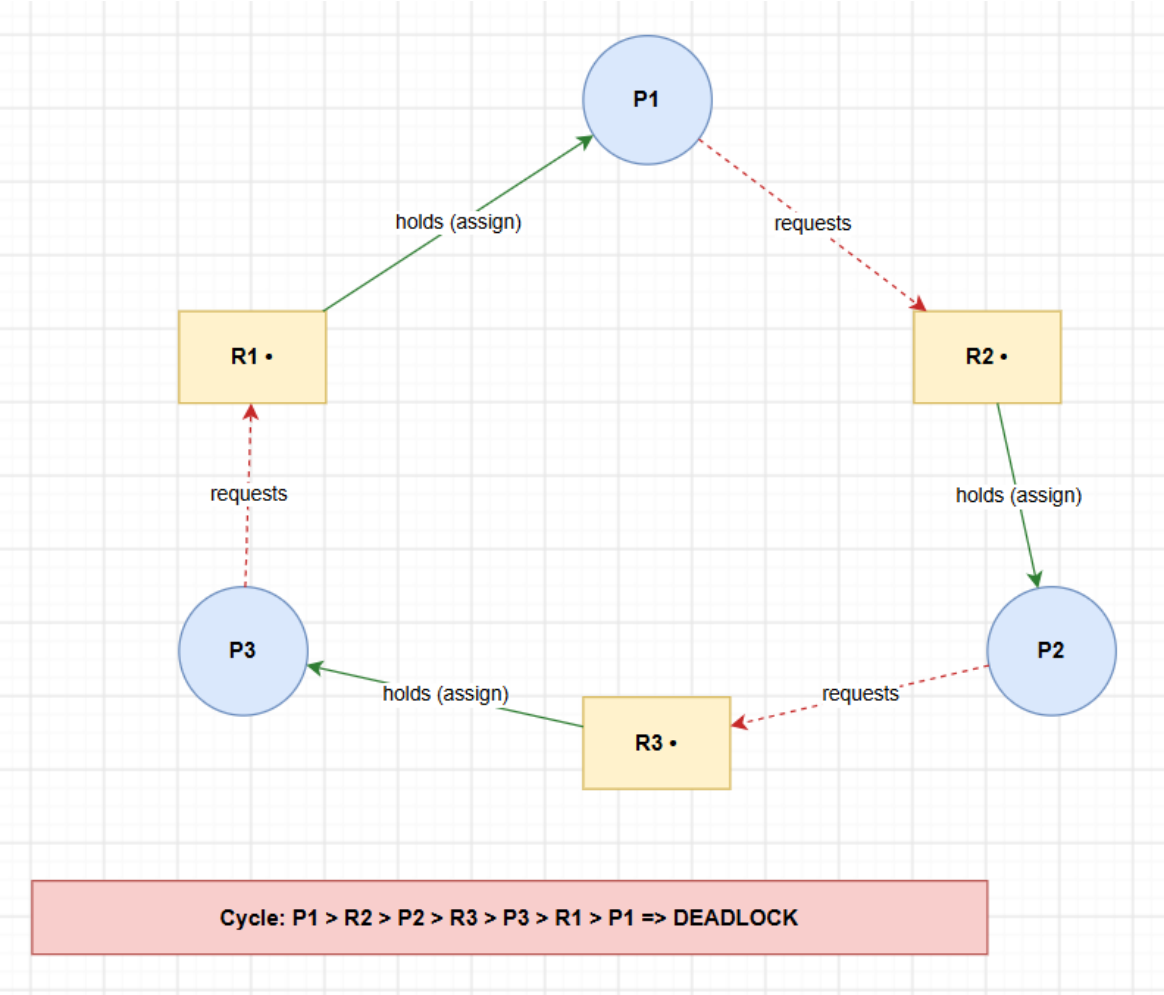
d. **Safe sequence: ⟨P2, P4, P5, P1, P3⟩** (other valid sequences exist, for example ⟨P2, P4, P1, P3, P5⟩).



# Question 4: Deadlock Detection – Resource Allocation Graph

**Assumption:** Each resource R1, R2, R3 has a single instance.

**a. RAG (described in text; draw.io: Q4_RAG.drawio)**




- Processes are circles: P1, P2, P3. Resources are rectangles with one dot: R1, R2, R3.
- **Assignment edges** (resource → process): R1 → P1, R2 → P2, R3 → P3.
- **Request edges** (process → resource): P1 → R2, P2 → R3, P3 → R1.

The resulting cycle is:

**P1 → R2 → P2 → R3 → P3 → R1 → P1**

**b. Deadlock determination**

The graph contains a cycle, and since every resource has a single instance, a cycle is both necessary and sufficient for deadlock. **The system is deadlocked.** All four Coffman conditions hold:

- Mutual exclusion: each resource is held by one process only.

- Hold and wait: every process holds one resource while requesting another.
- No preemption: resources are released only voluntarily.
- Circular wait: P1 waits for P2, P2 waits for P3, P3 waits for P1.

**c. Recommended strategy: Recovery (with prevention for the future)**

Because the deadlock has already occurred, I recommend **recovery by terminating or preempting one process** (for example, abort P3, the process whose rollback costs least, or preempt R3 from P3 and roll it back). Aborting P3 releases R3, so P2 gets R3 and finishes, then releases R2 for P1. The cycle is broken, and the remaining processes complete.

*Justification:* Recovery requires no advance knowledge of resource needs, and it adds no overhead during normal operation. It is cheap when deadlocks are rare. To stop recurrence, the OS can add **prevention by resource ordering** (R1 < R2 < R3, processes may request only in increasing order). P3 requesting R1 while holding R3 would then be rejected, which eliminates circular wait.

# Question 5: CPU Utilization – Multiprogramming Level

**Formula:** CPU Utilization = $1 - p^n$, with p = 0.8 (80% I/O wait).

**a. n = 4:** $1 - 0.8^4 = 1 - 0.4096 = 0.5904 \rightarrow$ **59.04%**

**b. n = 6:** $1 - 0.8^6 = 1 - 0.262144 = 0.7379 \rightarrow$ **73.79%**

| n | $p^n$ | Utilization |
|---|---|---|
| 4 | 0.4096 | 59.04% |
| 6 | 0.2621 | 73.79% |
| 8 (for reference) | 0.1678 | 83.22% |

**c. Interpretation**

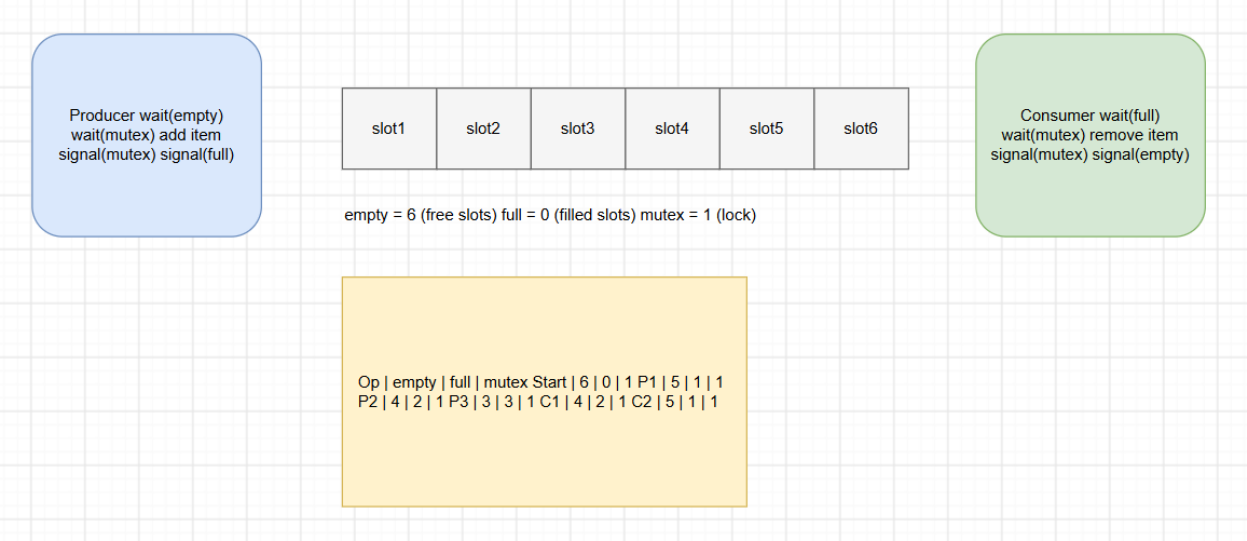
Increasing the degree of multiprogramming from 4 to 6 raises CPU utilization by about **14.75 percentage points** (roughly 25% relative improvement). With more processes in memory, the probability that all of them are waiting for I/O at the same time drops, so the CPU is idle less often. Hence **n = 6 is the better configuration.** Gains diminish (each extra process helps less than the last), and a very high n can cause memory pressure, thrashing and more context switching, so the degree of multiprogramming should be raised only as far as memory allows.

# Question 6: Producer–Consumer with Semaphores

**Setup:** Buffer size N = 6. Semaphores: **empty = 6** (free slots), **full = 0** (filled slots), **mutex = 1** (binary lock). Initially the buffer is empty.

**Producer:** wait(empty) → wait(mutex) → add item → signal(mutex) → signal(full) **Consumer:** wait(full) → wait(mutex) → remove item → signal(mutex) → signal(empty)

**a. Semaphore values after each operation (draw.io: Q6_Producer_Consumer.drawio)**




| Operation | empty | full | mutex | Items in buffer |
|---|---|---|---|---|
| Initial | 6 | 0 | 1 | 0 |
| Produce #1 | 5 | 1 | 1 | 1 |
| Produce #2 | 4 | 2 | 1 | 2 |
| Produce #3 | 3 | 3 | 1 | 3 |
| Consume #1 | 4 | 2 | 1 | 2 |
| Consume #2 | 5 | 1 | 1 | 1 |

While an operation is inside its critical section, mutex is temporarily 0 (for example, during Produce #1: empty = 5, full = 0, mutex = 0). It returns to 1 on signal(mutex), and full is incremented afterwards. The table shows the values at the end of each operation.

### b. How semaphores prevent race conditions

- **mutex** gives mutual exclusion: wait(mutex) is atomic, so only one process can modify the buffer and its in/out pointers at a time.
- **empty** blocks a producer when the buffer is full, so no overflow occurs.
- **full** blocks a consumer when the buffer is empty, so no underflow occurs.

- Because wait() and signal() are indivisible, no interleaving can corrupt the shared counters or pointers.

**c. If mutex is removed**

Two producers can pass wait(empty) together (empty = 6 allows both) and both read the same in index. Both write to the same slot, so one item is **overwritten and lost**, and in may be incremented only once while full is signalled twice. Similarly, two consumers may read the same item. The count of items and the buffer state become inconsistent. This is a **race condition on the shared buffer**. The empty/full semaphores only count slots; they do not protect access to the buffer structure, so mutex is essential.

# Question 7: Process Synchronization – Critical Section

**a. How the race condition occurs**

X = X + 1 is not atomic. It compiles to three steps: **LOAD** X into a register, **ADD** 1, **STORE** the register back to X. If a context switch happens between the load and the store, an update is lost.

Example with X = 100:

| Step | P1 | P2 | X |
|---|---|---|---|
| 1 | LOAD R1 ← X (100) | | 100 |
| 2 | | LOAD R2 ← X (100) | 100 |
| 3 | R1 = 101 | R2 = 101 | 100 |
| 4 | STORE X ← 101 | | 101 |
| 5 | | STORE X ← 101 | 101 |

Two increments were executed, but X increased only by 1. This is a **lost update**. Repeated across 2 × 1000 operations, the final value of X is unpredictable. It would lie somewhere between 102 and 2100 (typically below 2100).

**b. Expected value with proper synchronization**

Total increments = 2 × 1000 = 2000, so X = 100 + 2000 = **2100**.

**c. Solutions**

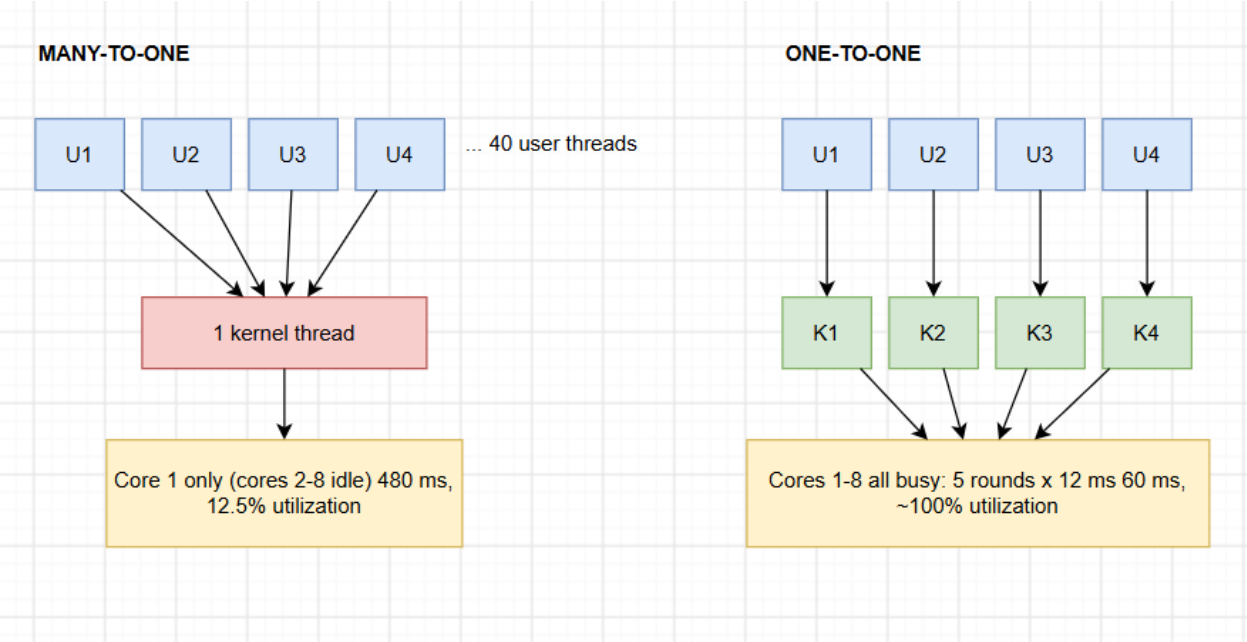
- **Hardware-based:** the atomic **Test-and-Set** instruction (or Compare-and-Swap). The lock is acquired with while (TestAndSet(&lock));, then the increment, then lock = false. The test and the set happen indivisibly, so only one process enters the critical section.

- **Software-based: Peterson's algorithm** using flag[i] and turn. Process i sets flag[i] = true; turn = j; and waits while flag[j] && turn == j. It satisfies mutual exclusion, progress and bounded waiting for two processes.
-

# Question 8: Thread Scheduling and Performance

**Given:** 40 user-level threads, 8 CPU cores, 12 ms CPU time per thread, negligible I/O. Total work = 40 × 12 = **480 ms**.

**a. Many-to-One vs One-to-One (draw.io: Q8_Threading_Models.drawio)**




| Feature | Many-to-One | One-to-One |
|---|---|---|
| Mapping | Many user threads → 1 kernel thread | Each user thread → its own kernel thread |
| Parallelism on multicore | None (one kernel thread runs on one core at a time) | Full, threads run on different cores |
| Blocking system call | Blocks the entire process | Blocks only that thread |
| Thread management | Fast, in user space | Slower; kernel involvement and more overhead |
| Scalability | Poor | Good (limited by kernel thread limits) |

**b. CPU utilization estimate**

- **Many-to-One:** Only 1 core is used. Completion time = 40 × 12 = **480 ms**. Utilization of the 8 cores = 1/8 = **12.5%**.
- **One-to-One:** The 40 threads are spread over 8 cores: 40 / 8 = 5 rounds of 12 ms, so completion time = 5 × 12 = **60 ms**. Utilization ≈ **100%** (ideal, ignoring kernel overhead).

Speedup = 480 / 60 = **8×**.

**c. Recommendation**

Use the **One-to-One model**. The workload is CPU-bound and the machine has 8 cores, so true parallelism is needed. Many-to-One would leave seven cores idle and would take 8× longer. One-to-One's extra overhead of creating 40 kernel threads is small compared with that gain. (A Many-to-Many model is a possible alternative if thousands of threads were needed.)

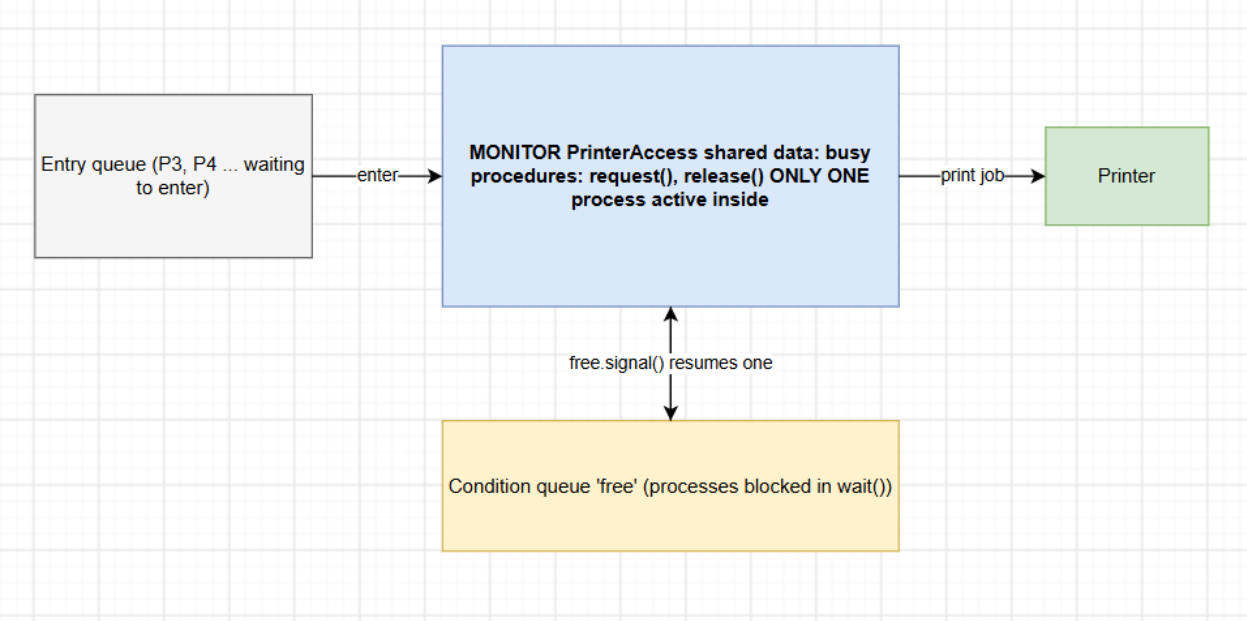
# Question 9: Monitor Synchronization

**Scenario:** A monitor controls access to a shared printer.

**a. Mutual exclusion**

A monitor is a high-level construct that bundles shared data and the procedures that operate on it. The language/runtime guarantees that **only one process can be active inside the monitor at any time**. Other processes calling a monitor procedure are queued at the monitor entry until the active process leaves or waits. The programmer does not write explicit lock code, which avoids errors such as forgetting a `signal(mutex)`.

```
monitor PrinterAccess {
  boolean busy = false;
  condition free;
  procedure request() { if (busy) free.wait(); busy = true; }
  procedure release() { busy = false; free.signal(); }
}
```




### b. Condition variables – wait() and signal()

- **x.wait():** the calling process is suspended and placed in the queue of condition x, and it releases the monitor so another process can enter. It is used when the printer is busy.
- **x.signal():** resumes exactly one process waiting on x (if any). It is used when the printer becomes free. If no process is waiting, the signal has no effect.

### c. What happens if signal() is omitted after a print job completes

Processes that called free.wait() are never woken, even though the printer is free. They remain blocked **indefinitely**, so the printer sits idle while jobs wait (a lost wakeup). This produces **starvation or deadlock-like behaviour**: system throughput drops, and the waiting processes never complete their jobs. Every release must therefore be followed by signal().

## Question 10: Deadlock Prevention vs Avoidance

**Scenario:** Four processes (P1–P4) compete for two reusable resources R1 and R2.

### a. Deadlock prevention

Prevention ensures that at least one of the four necessary (Coffman) conditions can never hold:

| Condition eliminated | How |
|---|---|
| Mutual exclusion | Make resources shareable (e.g., spooling a printer). This is not possible for all resources. |

| Condition eliminated | How |
|---|---|
| Hold and wait | A process must request all its resources at once (R1 and R2 together), or release everything before requesting more. |
| No preemption | If a request cannot be granted, the process releases what it holds, or the OS preempts the resources. |
| Circular wait | Impose a total order R1 < R2, and every process must request in that order. A cycle can never form. |

**Example:** With the ordering R1 < R2, if P1 holds R2 and wants R1, the request is rejected. No cycle is possible among P1–P4.

**b. Avoidance vs prevention (example)**

Avoidance (Banker's Algorithm) allows all four conditions to exist but grants a request **only if the resulting state is safe**. It needs advance knowledge of each process's maximum need.

Example: R1 and R2 have one instance each. P1 and P2 each need both. P1 holds R1. P2 now requests R2, which is free.

- **Prevention** (request all at once): P2 would have been forced to request R1 and R2 together and would simply wait for both.
- **Avoidance:** The OS tests the state if R2 goes to P2. P1 would still need R2 and P2 would still need R1, so neither can finish. This is an **unsafe state**, so the request is **delayed** even though R2 is free. P1 is allowed to finish first, and then P2 proceeds.

**c. Comparison**

| Aspect | Prevention | Avoidance |
|---|---|---|
| Approach | Structurally removes one Coffman condition | Dynamically checks each request for safe state |
| Advance information | Not needed | Needs maximum resource needs |
| Resource utilization | Low (resources held idle, or requested early) | Higher; resources granted whenever safe |
| Run-time overhead | Low | High (safety check on every request) |
| System performance | Possible low throughput, starvation | Better throughput, but $O(m \cdot n^2)$ cost per check |
| Deadlock guarantee | Yes | Yes (if the maximum claims are correct) |

**Conclusion:** Prevention is simple and predictable but wastes resources. Avoidance uses resources better but costs more computation and requires known maximum demands. The choice depends on how predictable the workload is.

## Summary of Key Results

| Q | Key result |
|---|---|
| 1 | SJF: avg WT = 5 ms, avg TAT = 10.5 ms (FCFS: 11 / 16.5) |
| 2 | RR q=4: avg WT = 23, avg TAT = 34.5; q=10: 22.75 / 34.25 |
| 3 | Safe state; sequence P2, P4, P5, P1, P3 |
| 4 | Cycle exists → deadlock; recover by terminating/preempting P3 |
| 5 | n=4: 59.04%; n=6: 73.79% |
| 6 | Semaphores end at empty = 5, full = 1, mutex = 1 |
| 7 | Correct final X = 2100 |
| 8 | Many-to-One 12.5% utilization (480 ms); One-to-One ≈100% (60 ms) |
| 9 | Monitor ensures one active process; missing signal() leaves waiters blocked |
| 10 | Prevention breaks a condition; avoidance keeps the state safe |

## RUBRICS FOR CALCULATION: Analytical Problem Solving

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 20% | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection | 20% | Selects most appropriate method/formula with strong justification | Selects correct method with minor errors | Inappropriate or partially correct method | Unable to select method |
| Solution Process | 25% | Logical, systematic steps with correct approach throughout | Mostly correct steps with minor mistakes | Disorganized steps; multiple errors | No clear process |
| Accuracy of Calculation | 20% | All calculations accurate; correct final answer | Minor calculation errors | Frequent errors; incorrect final answer | No valid calculations |
| Interpretation of Results | 15% | Clearly interprets results with units and engineering relevance | Basic interpretation with minor omissions | Limited or unclear interpretation | No interpretation |

**Marks Evaluation:**

| Question No. | Problem Understandin (20%) | Method Selection(20%) | Solution Process(25%) | Accuracy of Calculation (20%) | Interpretation of Results (15%) | Total Marks (100) |
|---|---|---|---|---|---|---|
| Q1 | | | | | | |
| Q2 | | | | | | |
| Q3 | | | | | | |
| Q4 | | | | | | |
| Q5 | | | | | | |
| Q6 | | | | | | |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Q7** | | | | | | |
| **Q8** | | | | | | |
| **Q9** | | | | | | |
| **Q10** | | | | | | |
| | | | | | **Overall Total (100)** | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |